UK PLANNING INTELLIGENCE

# Every policy in every Local Plan, summarised *automatically*

Every UK council publishes a Local Plan, which runs to hundreds of pages and up to a hundred-plus policies and gets revised at every stage on the way to adoption. I built the pipeline that reads them all, summarising 700+ plans every week fully automated, to power LandGPT, the planning chatbot of a UK planning-intelligence company. Weeks of reading per plan went down to minutes, at about a quarter of the original LLM cost.

**700+**
PLANS PROCESSED WEEKLY

**75%** LLM COST CUT

**100+** CONCURRENT QUERIES PER PLAN

**Minutes** PER PLAN, DOWN FROM WEEKS

ADAM MURPHY

AI ENGINEER · EXPERT-VETTED ON UPWORK

THE SITUATION

## Weeks of reading, multiplied by every council

A Local Plan is a council's rulebook for development, setting out what can be built, where and on what terms. Each one runs to hundreds of pages and up to a hundred-plus policies, and each exists at up to three lifecycle stages (Preferred Options, Pre-Submission and Adoption) with the policies shifting at every one. If you need to know what councils will permit, that's weeks of reading per plan, multiplied across every council in the country.

A UK planning-intelligence company was building LandGPT, a chatbot that answers planning questions from the policies themselves, and it needed a clean summary of every policy in every plan, kept up to date as plans moved through adoption. I built that pipeline over a 758-hour contract, which was the first major build in what became an eighteen-month working relationship.

WHAT I BUILT

## Upload each plan once, query it a hundred times

The pipeline runs on Gemini 1.5 Pro through Vertex AI and leans hard on context caching. Each plan is uploaded and cached once, then up to 100+ per-policy queries run concurrently against the cache, so you pay for the plan's hundreds of pages once rather than once per policy. A two-stage chain does the work. Stage one pulls out the complete policy list as structured output and retries until it parses, and stage two summarises every policy on that list in parallel.

01
**Local Plan in**
Council PDF uploaded and context-cached once, per lifecycle stage

→

02
**Policies extracted**
Structured policy list, retried until it parses cleanly

→

03
**Summarised in parallel**
100+ concurrent per-policy queries against the cache

→

04
**Stored & served**
Postgres upserts, S3 audit copies, straight into LandGPT

Around the model sits the plumbing that turns this from a demo into a weekly production job. Results are upserted into Postgres so re-runs are safe, every run leaves a CSV audit copy in S3, and failures raise Slack alerts instead of passing silently. Before I took it on, the pipeline sent the whole plan to GPT-4o again for every policy through OpenAI's Batch API, so a long plan was paid for once per policy. With the plan cached once on Gemini, every policy question reads it at a quarter of the normal input price, and LLM cost per plan came down about 75%.

| 700+ | 100+ | 3 | 75% |
|---|---|---|---|
| PLANS PROCESSED WEEKLY | CONCURRENT QUERIES PER PLAN | LIFECYCLE STAGES COVERED | LLM COST REDUCTION |

THE HARD PARTS

# What it takes to read a planning system weekly

### The cache had a minimum, and short plans missed it

Vertex AI context caching only accepts documents of 32,768 tokens or more, and plenty of Local Plans fall short. I measured how Gemini tokenises whitespace (35 spaces to the token) and padded short plans up to the threshold. It's not elegant, but it's measured, and it let every plan share one caching architecture and one cost model.

### Gemini's hard ceilings

The model has hard limits of 1,000 pages per PDF and 50MB per file, and the longest Local Plans break them. The pipeline spots oversized documents and splits them up automatically, so the largest plans go through the same two-stage chain as a 200-page one and nobody has to split PDFs by hand.

### A summary is only as good as the policy list

A malformed policy list poisons everything downstream, because summaries get orphaned and counts drift and nobody notices until someone queries a policy. So extraction returns a structured, numbered policy list and retries automatically whenever the output fails to parse, which means nothing downstream ever runs against a malformed list. The easy mistake would have been polished summaries sitting on a shaky index.

### Nobody watches a weekly job

At 700+ plans a week, the pipeline has to notice its own failures. Postgres upserts make every re-run safe to repeat, each run writes a CSV audit copy to S3, and anything that breaks raises a Slack alert, because it's silent failure rather than model quality that usually kills unattended pipelines.

THE RESULTS

| | |
|---|---|
| THROUGHPUT | **700+** Local Plans processed weekly, fully automated |
| COST | About **75%** lower LLM cost per plan from context caching on Gemini 1.5 Pro, replacing a GPT-4o batch job that resent the whole plan for every policy |
| COVERAGE | All **700+** plans across three lifecycle stages, up to **100+** policies summarised per plan |
| ENGAGEMENT | **5.0★** Upwork contract · **758 hours** · $37,891.66 · Dec 2023 to Aug 2024 |

I haven't named the client company here, but the product name, the review and the contract figures are public on Upwork.

> THE CLIENT, IN HIS OWN WORDS
>
> *"It was great to work with Adam – great communicator and great dev!"*
>
> **JOS PINK** · MANAGING DIRECTOR (5.0★ UPWORK REVIEW)

ABOUT ADAM

I'm a freelance Senior AI Engineer, **Expert-Vetted on Upwork (top 1%)**, with 100% Job Success across 75+ projects, $500K+ earned and 6,000+ hours over seven years. I build production LLM systems for regulated industries like insurance, finance and law.

Upwork profile ↗ · github.com/codeananda ↗